zl_robot 仿真平台使用说明

本文说明如何启动 zl_robot MuJoCo 仿真平台，并以 `heart` 模型为例完成新 RL 模型的加载和切换调用。

1. 目录说明

当前发布目录包含以下关键文件和目录：

```
scripts/
  run.sh        # 启动脚本
  shutdown.sh   # 关闭脚本

docs/
  RobotIntent发布命令速查.md
  robot_rl_manager模型适配教程.md
  example/
    heart.py
    heart.yaml
    t4_heart_mjlab.onnx
    t4_heart_mjlab.npz
```

2. 启动仿真程序

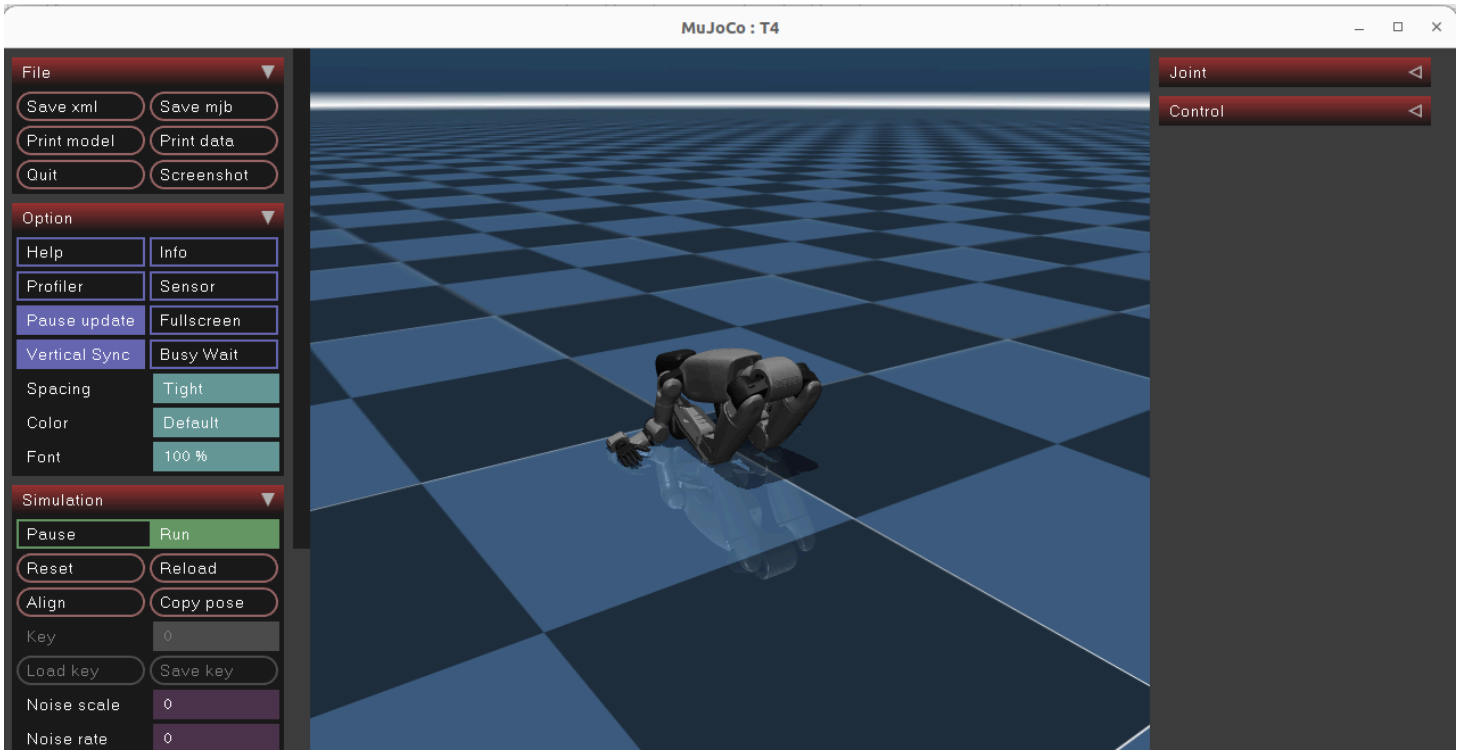
打开终端，进入当前发布目录，后续称为运行终端：

```
cd ./zl_robot_release
```

启动程序：

```
./scripts/run.sh
```

启动后会弹出 MuJoCo 仿真程序，并自动加载机器人模型。刚加载完成时，机器人初始处于坍塌状态。



3. 发送 ready 指令

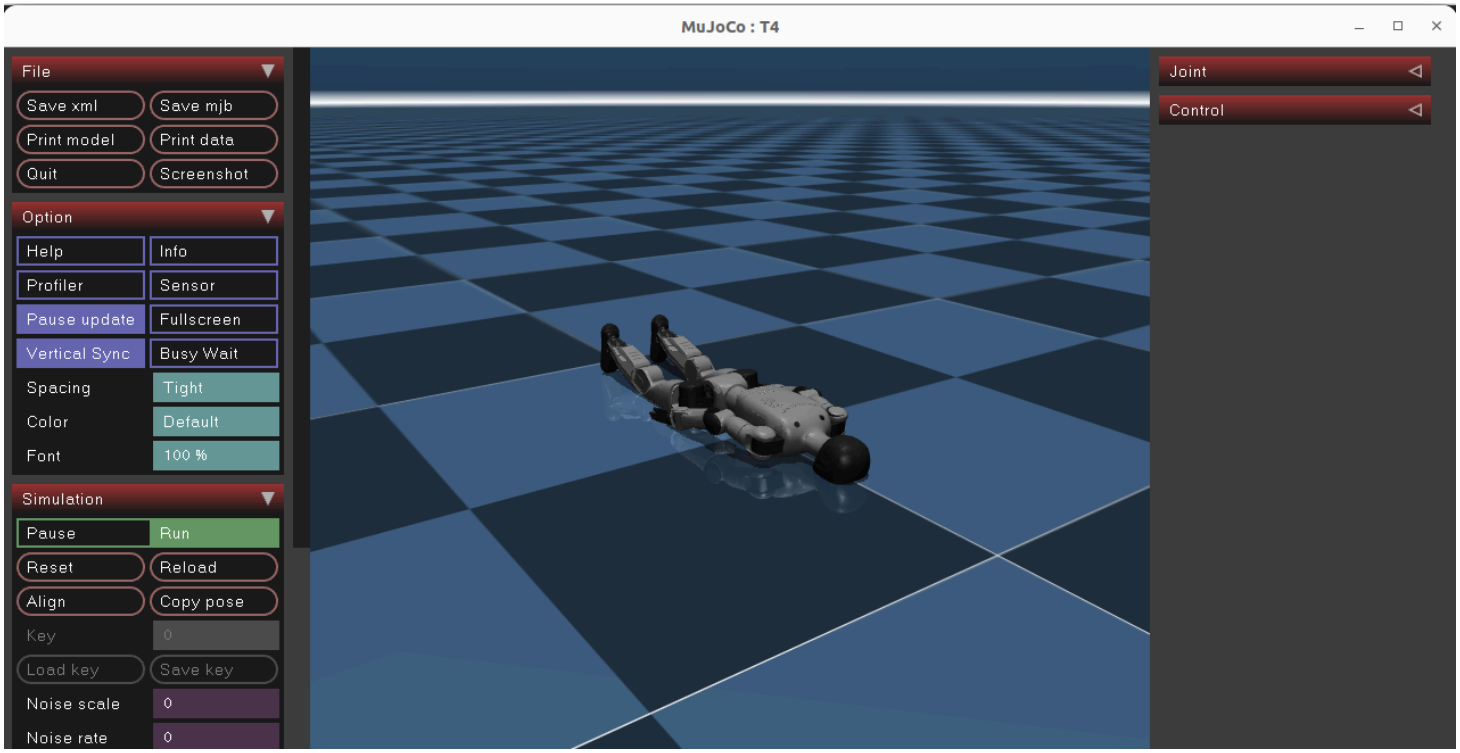
另开一个终端，进入同一发布目录并加载 ROS 环境，后续称为调试终端：

```
cd ./zl_robot_release
source install/setup.bash
```

在调试终端中发送 `ready` 指令：

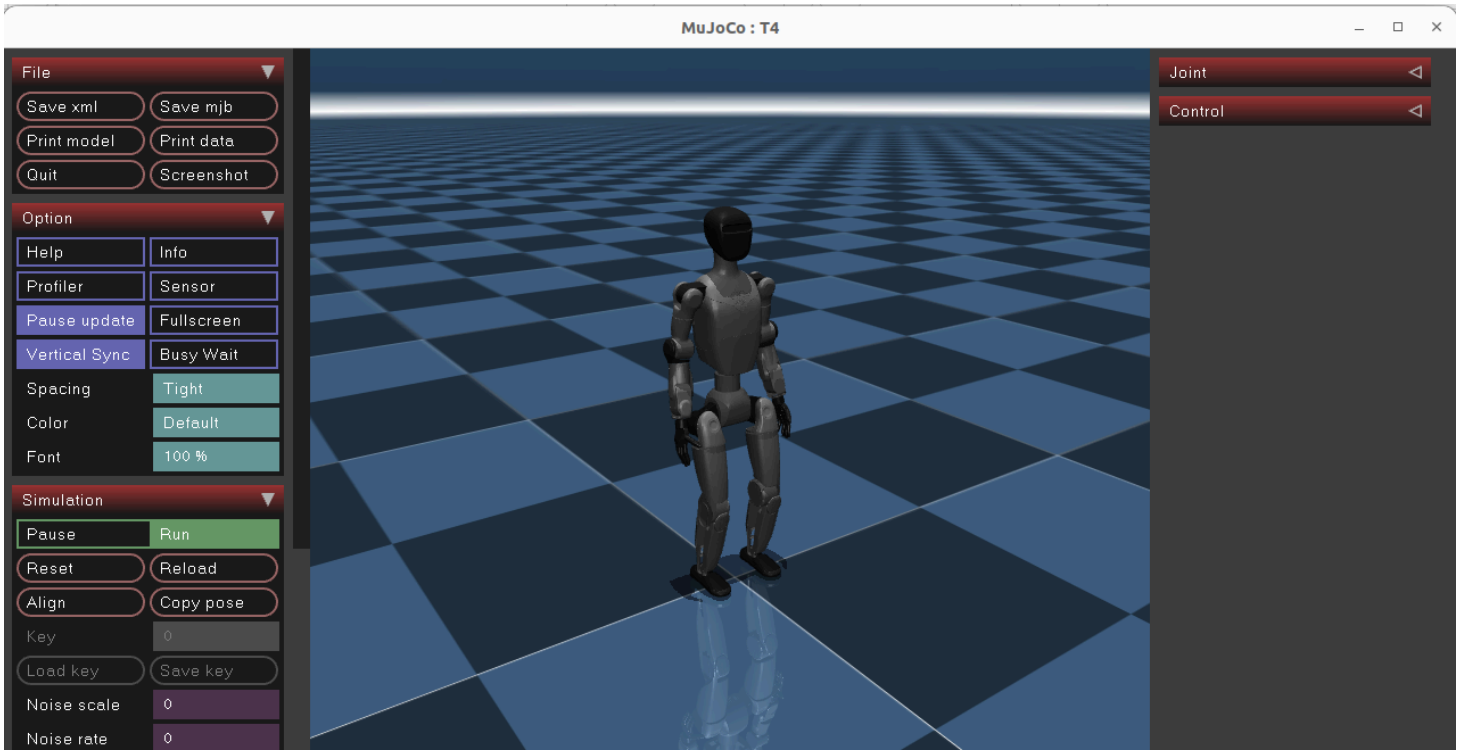
```
ros2 topic pub /robot/input/intent robot_interfaces/msg/RobotIntent \
  "{source: 'teleop', intent_type: 'status', command: 'ready', \
  priority: 80, timeout_sec: 0.5, require_ack: false}" --once
```

此时仿真界面中的倒地机器人会慢慢进入站立准备状态。等待运行终端和语音播报提示“准备就绪”。



4. 点击 reset 进入稳定初始站立状态

在 MuJoCo 仿真程序界面点击 `reset` 按钮。机器人会进入初始状态，此时已经能够稳定站立。

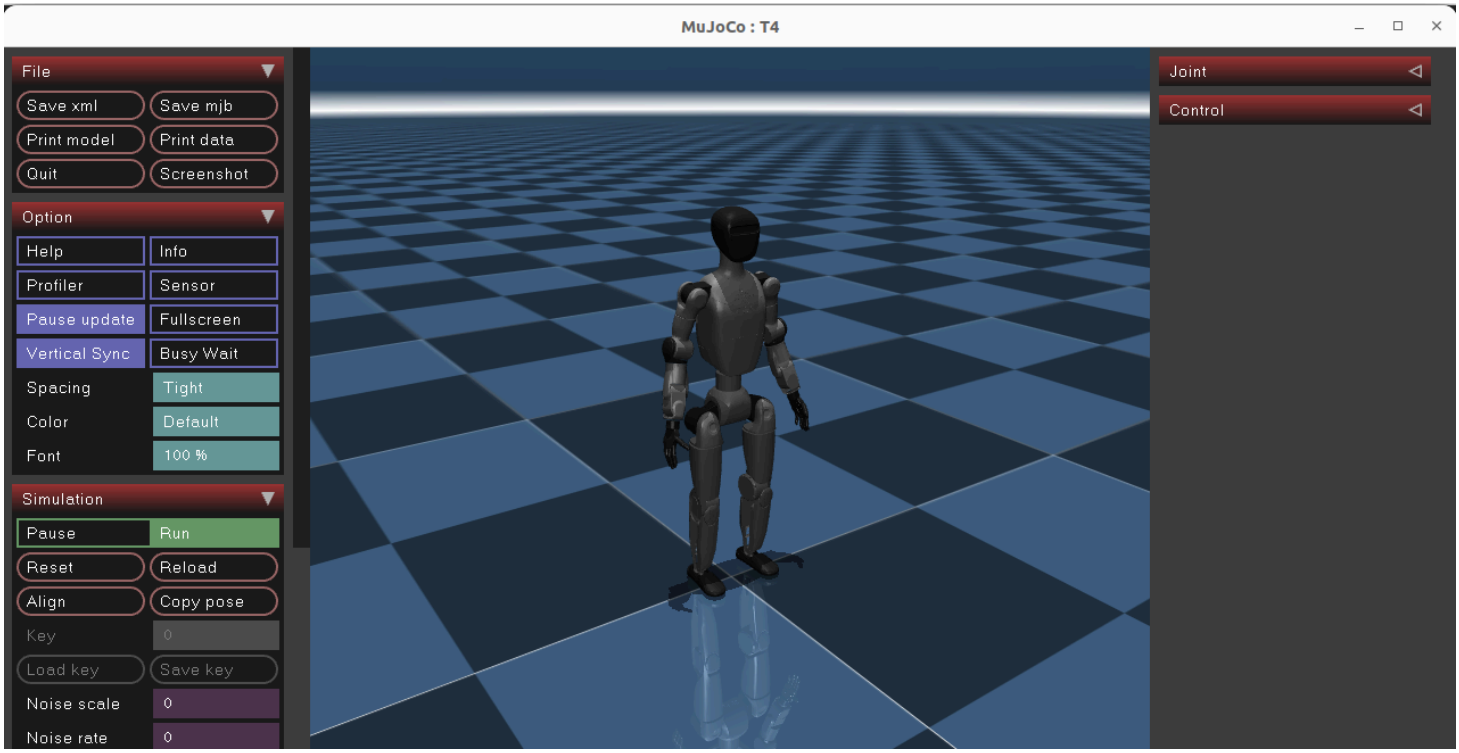


5. 发送 walk 指令进入行走模型

继续在调试终端发送 `walk` 指令：

```
ros2 topic pub /robot/input/intent robot_interfaces/msg/RobotIntent \
  "{source: 'teleop', intent_type: 'status', command: 'walk', \
  priority: 80, timeout_sec: 0.5, require_ack: false}" --once
```

机器人进入行走模型。注意所有模型切换动作，都需要机器人处于行走模型才能切换。



如果需要发布连续运动速度，可继续参考 `docs/RobotIntent发布命令速查.md` 中的 `motion` 示例。

6. 准备新模型适配文件

新模型适配所需文件请参考 `docs/robot_rl_manager模型适配教程.md` 。一个可加载的模型通常需要以下文件：

- 模型文件，例如 `.onnx`
- 模型配置文件，例如 `.yaml`
- 模型适配器文件，例如 `.py`

本文以 `heart` 模型为例，示例文件已经放在：

```
./zl_robot_release/docs/example/
```

对应文件为：

```
./zl_robot_release/docs/example/t4_heart_mjlab.onnx
./zl_robot_release/docs/example/heart.yaml
./zl_robot_release/docs/example/heart.py
```

7. 模型未加载时直接切换 heart

在加载 `heart` 模型之前，先尝试切换到 `heart`，在调试终端发送 `switch_model` 服务请求：

```
ros2 service call /robot/rl_manager/switch_model robot_interfaces/srv/SwitchRlModel \
  "{model_name: heart}"
```

此时运行终端中会提示 `heart` 模型尚未加载，仿真中的机器人不会执行动作。

```
zoomlion@zoomlion-virtual-machine:~/work/99_robot/zl_robot_ota/releases$ ros2 service call /robot/rl_manager/switch_model robot_i
nterfaces/srv/SwitchRlModel   "{model_name: heart}"
requester: making request: robot_interfaces.srv.SwitchRlModel_Request(model_name='heart')

response:
robot_interfaces.srv.SwitchRlModel_Response(success=False, message="'unknown RL model: heart'")
```

8. 加载 heart 模型

在调试终端中发送 `load_model` 服务请求加载 `heart` 模型：

```
ros2 service call /robot/rl_manager/load_model robot_interfaces/srv/LoadRlModel \
  "{model_name: heart, model_path: ./docs/example/t4_heart_mjlab.onnx, \
  model_type: onnx, config_path: ./docs/example/heart.yaml, \
  adapter_path: ./docs/example/heart.py}"
```

加载成功后，调试终端中会出现模型加载成功的提示信息。

```
zoomlion@zoomlion-virtual-machine:~/work/99_robot/zl_robot_ota$ ros2 service call /robot/rl_manager/load_model robot_interfaces/srv/LoadRlModel \
  "{model_name: heart, model_path: ./docs/example/t4_heart_mjlab.onnx, model_type: onnx, config_path: ./docs/example/heart.yaml, adapter_path: ./docs/example/heart.py}"
requester: making request: robot_interfaces.srv.LoadRlModel_Request(model_name='heart', model_path='./docs/example/t4_heart_mjlab.onnx', model_type='onnx', config_path='./docs/example/heart.yaml', adapter_path='./docs/example/heart.py')

response:
robot_interfaces.srv.LoadRlModel_Response(success=True, message='model loaded: heart')
```

9. 切换 heart 模型并执行动作

模型加载成功后，在调试终端中再次发送 `switch_model` 服务请求：

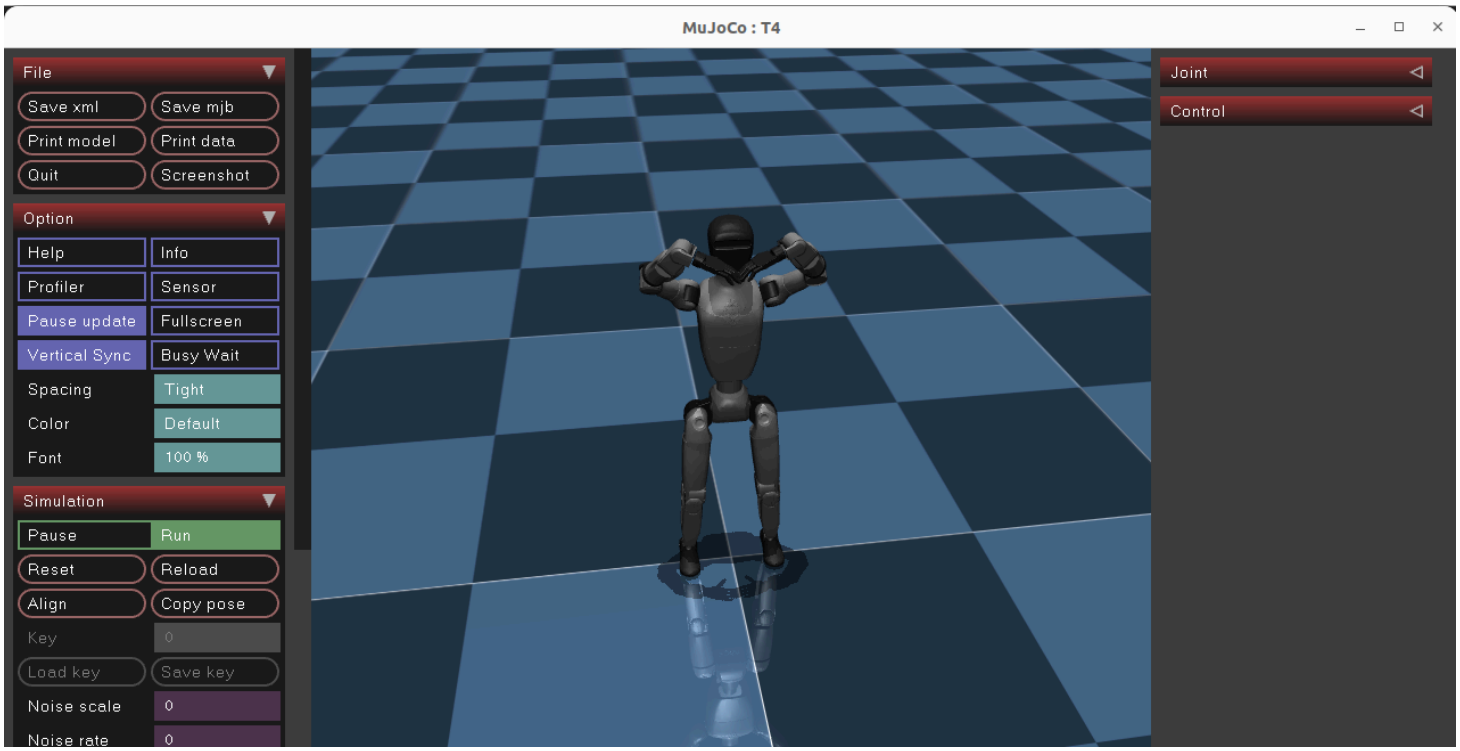
```
ros2 service call /robot/rl_manager/switch_model robot_interfaces/srv/SwitchRlModel \
  "{model_name: heart}"
```

此时运行终端中会提示 `heart` 模型切换成功。

```
zoomlion@zoomlion-virtual-machine:~/work/99_robot/zl_robot_ota$ ros2 service call /robot/rl_manager/switch_model robot_interfaces/srv/SwitchRlModel   "{model_name: heart}"
requester: making request: robot_interfaces.srv.SwitchRlModel_Request(model_name='heart')

response:
robot_interfaces.srv.SwitchRlModel_Response(success=True, message='active model: heart')
```

仿真中的机器人会执行一次比心动作。



至此，模型加载和调用流程结束。

10. 进入阻尼状态

在调试终端中发送 `damp` 指令，机器人进入阻尼状态：

```
ros2 topic pub /robot/input/intent robot_interfaces/msg/RobotIntent \
  "{source: 'teleop', intent_type: 'status', command: 'damp', \
  priority: 80, timeout_sec: 0.5, require_ack: false}" --once
```

11. 回到待机状态

在调试终端中发送 `reinit` 指令，机器人进入待机状态：

```
ros2 topic pub /robot/input/intent robot_interfaces/msg/RobotIntent \
  "{source: 'teleop', intent_type: 'status', command: 'reinit', \
  priority: 80, timeout_sec: 0.5, require_ack: false}" --once
```

回到待机状态后，可以重新进行一次模型切换操作。模型加载操作与机器人当前状态无关，已经加载过的模型不需要重复加载。

12. 关闭程序

使用结束后，可在当前发布目录执行关闭脚本：

```
./scripts/shutdown.sh
```

如果需要强制关闭残留进程，可执行：

```
./scripts/shutdown.sh --force
```